

```

# !pip install transformers torch

from transformers import AutoTokenizer, AutoModel
import torch
from torch.nn.functional import cosine_similarity

def get_bertimbau_embeddings(text, model, tokenizer):
    inputs = tokenizer(text, padding=True, truncation=True, max_length=512,
return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)
    embeddings = outputs.last_hidden_state.mean(dim=1)
    return embeddings

def analyze_document(document_text, model, tokenizer):
    max_chars = 512
    sections = [document_text[i:i+max_chars] for i in range(0, len(document_text),
max_chars)]
    section_embeddings = [get_bertimbau_embeddings(section, model, tokenizer) for
section in sections]
    document_embedding = torch.mean(torch.stack(section_embeddings), dim=0)
    return document_embedding

# Textos de exemplo
texto1 = "Potencial da aplicação de técnicas de Processamento de Linguagem Natural
(PLN) e de Machine Learning (ML) no processo de categorização para a Análise de Domínio
(AD). "
texto2 = "A Dimensão Aplicada da Organização do Conhecimento no Campo da Inteligência
Artificial."

# Carrega o modelo e tokenizer (fora do loop para evitar recarregamento)
tokenizer = AutoTokenizer.from_pretrained('neuralmind/bert-base-portuguese-cased')
model = AutoModel.from_pretrained('neuralmind/bert-base-portuguese-cased')

# Gera embeddings para os textos
embedding1 = analyze_document(texto1, model, tokenizer)
embedding2 = analyze_document(texto2, model, tokenizer)

# Calcula a similaridade de cosseno
similarity = cosine_similarity(embedding1, embedding2)

# Exibe os resultados
print("Dimensões do embedding 1:", embedding1.shape)
print("Dimensões do embedding 2:", embedding2.shape)
print("Similaridade entre os textos:", similarity.item())

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab and
restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public
models or datasets.
    warnings.warn(
tokenizer_config.json: 100%
 43.0/43.0 [00:00<00:00, 1.12kB/s]
config.json: 100%
 647/647 [00:00<00:00, 11.9kB/s]
vocab.txt: 100%
210k/210k [00:00<00:00, 2.55MB/s]

```

```
added_tokens.json: 100%
 2.00/2.00 [00:00<00:00, 39.3B/s]
special_tokens_map.json: 100%
112/112 [00:00<00:00, 2.41kB/s]
pytorch_model.bin: 100%
438M/438M [00:02<00:00, 231MB/s]
model.safetensors: 100%
438M/438M [00:02<00:00, 198MB/s]
Dimensões do embedding 1: torch.Size([1, 768])
Dimensões do embedding 2: torch.Size([1, 768])
Similaridade entre os textos: 0.7217277884483337
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100% 43.0/43.0 [00:00<00:00, 1.12kB/s]
config.json: 100% 647/647 [00:00<00:00, 11.9kB/s]
vocab.txt: 100% 210k/210k [00:00<00:00, 2.55MB/s]
added_tokens.json: 100% 2.00/2.00 [00:00<00:00, 39.3B/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 2.41kB/s]
pytorch_model.bin: 100% 438M/438M [00:02<00:00, 231MB/s]
model.safetensors: 100% 438M/438M [00:02<00:00, 198MB/s]
Dimensões do embedding 1: torch.Size([1, 768])
Dimensões do embedding 2: torch.Size([1, 768])
Similaridade entre os textos: 0.7217277884483337
```